דוח פרויקט: סוכן Deep Q-Learning למשחק Snake

2026-10-04

1. מבוא ומטרה

הסוכן הסופי מגיע לממוצע של **145.4 נקודות** ב-500 משחקים, פי 4.3 מהמימוש המקורי (33.5). הוא מגיע לזה אחרי **20 שניות אימון**, לעומת כשעה וחצי במקור.

מטרת הפרויקט היא ללמד מחשב לשחק Snake בעזרת **למידת חיזוק עמוקה (Deep Reinforcement Learning)**. הסוכן לא מקבל אף דוגמה של משחק נכון. הוא לומד רק מהתגמול המספרי שמגיע מהסביבה: 10+ כשהנחש אוכל, 10− כשהוא מת, ו-0 בכל צעד אחר.

העבודה נעשתה בשני שלבים:

1. **מימוש בסיסי** (DQN: `helper.py`, `model.py`, `agent.py`, `game.py`) קלאסי על משחק pygame יחיד. הוא עובד, אבל לאט, ונתקע על ממוצע של כ-33.
2. **מימוש משופר**: סביבה וקטורית שמריצה 64 משחקים במקביל, Double DQN עם רשת מטרה, ו-5 חיישנים חדשים שנותנים לנחש תמונה של השטח הפנוי סביבו.

הדוח מסביר את התיאוריה, כל קובץ וכל פונקציה, את הסיבה לכל החלטה, ואת התוצאות שנמדדו.

2. רקע תיאורטי

הפרויקט מממש את אלגוריתם **Double DQN**: רשת נוירונים שלומדת להעריך כמה נקודות עתידיות שווה כל פעולה בכל מצב.

2.1 למידת חיזוק ו-MDP

הבעיה מנוסחת כתהליך החלטה מרקובי (MDP). בכל צעד הסוכן רואה מצב s, בוחר פעולה a, מקבל תגמול r ועובר למצב חדש s'.

רכיב	בפרויקט
מצב (state)	וקטור של 11 או 16 חיישנים (פרק 6)
פעולות (actions)	3: ישר, פנייה ימינה, פנייה שמאלה (יחסית לכיוון הנוכחי)
תגמול (reward)	10+ אכילה, 10− מוות, 0 אחרת
סוף משחק	פגיעה בקיר, פגיעה בגוף, או רעב (יותר מ-100×אורך צעדים בלי אוכל)
מקדם הנחה γ	0.9 במקור, 0.95 במימוש החדש

הבחירה בפעולות יחסיות מונעת מהנחש להסתובב 180 מעלות לתוך עצמו, ומקטינה את מרחב הפעולות מ-4 ל-3.

2.2 Q-Learning

פונקציית הערך Q(s, a) היא סכום התגמולים המונחת שהסוכן יקבל אם יבצע את a במצב s ומשם ישחק באופן אופטימלי. היא מקיימת את משוואת בלמן:

$$Q(s, a) = r + \gamma \cdot \max_{a'} Q(s', a')$$

כשהמשחק נגמר (done) אין עתיד, והמטרה היא פשוט r.

2.3 Deep Q-Network (DQN)

מספר המצבים אפשריים עצום, ולכן אי אפשר לשמור טבלה. במקום זה רשת נוירונים מקבלת את וקטור המצב ומחזירה 3 ערכי Q, אחד לכל פעולה. הרשת מאומנת להקטין את ההפרש בין התחזית שלה לבין מטרת בלמן.

שלושה מנגנונים מייצבים את האימון:

- **Experience Replay:** כל מעבר (s, a, r, s′, done) נשמר בזיכרון, והאימון נעשה על מנות אקראיות ממנו. זה שובר את המתאם בין צעדים עוקבים ומאפשר ללמוד מכל דוגמה כמה פעמים.
- **ε-greedy:** בהסתברות ε הסוכן בוחר פעולה אקראית (חקירה), ואחרת את הפעולה עם ה-Q הגבוה (ניצול). ε יורד במהלך האימון.
- **Target Network:** עותק של הרשת שמתעדכן לאט ומשמש לחישוב המטרה. בלעדיו, המטרה זזה עם כל צעד של הרשת עצמה, והאימון נוטה להתבדר.

2.4 Double DQN

ה-max במשוואת בלמן בוחר את הערך הגבוה ביותר, ולכן נוטה לבחור בערכים שהוערכו ביתר בטעות (overestimation). Double DQN מפריד בין מי שבוחר למי שמעריך: הרשת הראשית בוחרת את הפעולה, ורשת המטרה מעריכה אותה.

$$a^* = \text{argmax}_{a'} Q_{\text{online}}(s', a') \qquad y = r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{target}}(s', a^*)$$

עדכון רשת המטרה הוא עדכון רך (soft update) אחרי כל צעד למידה, עם τ = 0.01:

$$\theta_{\text{target}} \leftarrow \tau \cdot \theta_{\text{online}} + (1 - \tau) \cdot \theta_{\text{target}}$$

3. המימוש המקורי ומגבלותיו

המימוש המקורי לומד, אבל מוגבל ל-39 צעדים בשנייה ולממוצע של כ-33. הקבצים נשארו בפרויקט בלי שינוי, כנקודת השוואה.

3.1 הקבצים והפונקציות

קובץ	פונקציה / מחלקה	מה היא עושה
`game.py`	`SnakeGameAI.__init__`	פותח חלון pygame בגודל 640×480 (לוח של 32×24 משבצות של 20 פיקסלים)
`game.py`	`reset`	נחש באורך 3 במרכז, פונה ימינה, ניקוד 0, אוכל חדש
`game.py`	`_place_food`	משבצת אקראית; אם היא על הנחש, קריאה רקורסיבית מחדש
`game.py`	`play_step(action)`	מזיז את הנחש, בודק מוות ואכילה, מצייר, ומחכה ל- `clock.tick(40)` . מחזיר (תגמול, סוף, ניקוד)
`game.py`	`is_collision(pt)`	פגיעה בקיר או בגוף (חיפוש ברשימה)
`game.py`	`_move(action)`	מתרגם [ישר, ימינה, שמאלה] לכיוון מוחלט בעזרת סדר השעון
`game.py`	`_update_ui`	ציור הנחש, האוכל והניקוד
`agent.py`	`Agent.__init__`	רשת 11→256→3, זיכרון `deque` של 100 אלף, טעינת `model/model.pth` אם קיים
`agent.py`	`get_state(game)`	11 חיישנים בוליאניים: סכנה ישר/ימינה/שמאלה, כיוון, מיקום האוכל
`agent.py`	`remember`	הוספת מעבר לזיכרון
`agent.py`	`train_short_memory`	צעד למידה על המעבר האחרון בלבד
`agent.py`	`train_long_memory`	בסוף כל משחק: צעד למידה על 1,000 מעברים אקראיים
`agent.py`	`get_action`	ε-greedy עם ε = 80 − מספר המשחקים
`agent.py`	`train()`	הלולאה הראשית; שומרת מודל בכל שיא חדש
`model.py`	`Linear_QNet`	שכבה נסתרת אחת עם `save` ;ReLU שומר משקלים, מספר משחקים ושיא
`model.py`	`QTrainer.train_step`	MSE מול מטרת בלמן, מחושבת בלולאת Python על כל דוגמה, עם אותה רשת עצמה
`helper.py`	`plot`	גרף חי של הניקוד והממוצע, עם `plt.pause(0.1)`

3.2 צווארי הבקבוק

1. **ציור בכל צעד:** `clock.tick(40)` מגביל את האימון לכ-39 צעדים בשנייה, גם כשאין מי שצופה.
2. **משחק אחד בלבד:** כל צעד מוסיף לזיכרון דוגמה אחת, והדוגמאות הסמוכות מתואמות מאוד.
3. `plt.pause(0.1)` אחרי כל משחק מוסיף עוד עיכוב.
4. **לולאת Python ב- `train_step` :** הרשת מופעלת בנפרד על כל אחת מ-1,000 הדוגמאות, במקום פעם אחת על כל ה-batch.
5. **אין רשת מטרה:** המטרה מחושבת עם אותה רשת שמתעדכנת, ולכן זזה בכל צעד.

6. **חקירה קצרה מדי:** הפעולות האקראיות נעלמות לגמרי אחרי 80 משחקים.

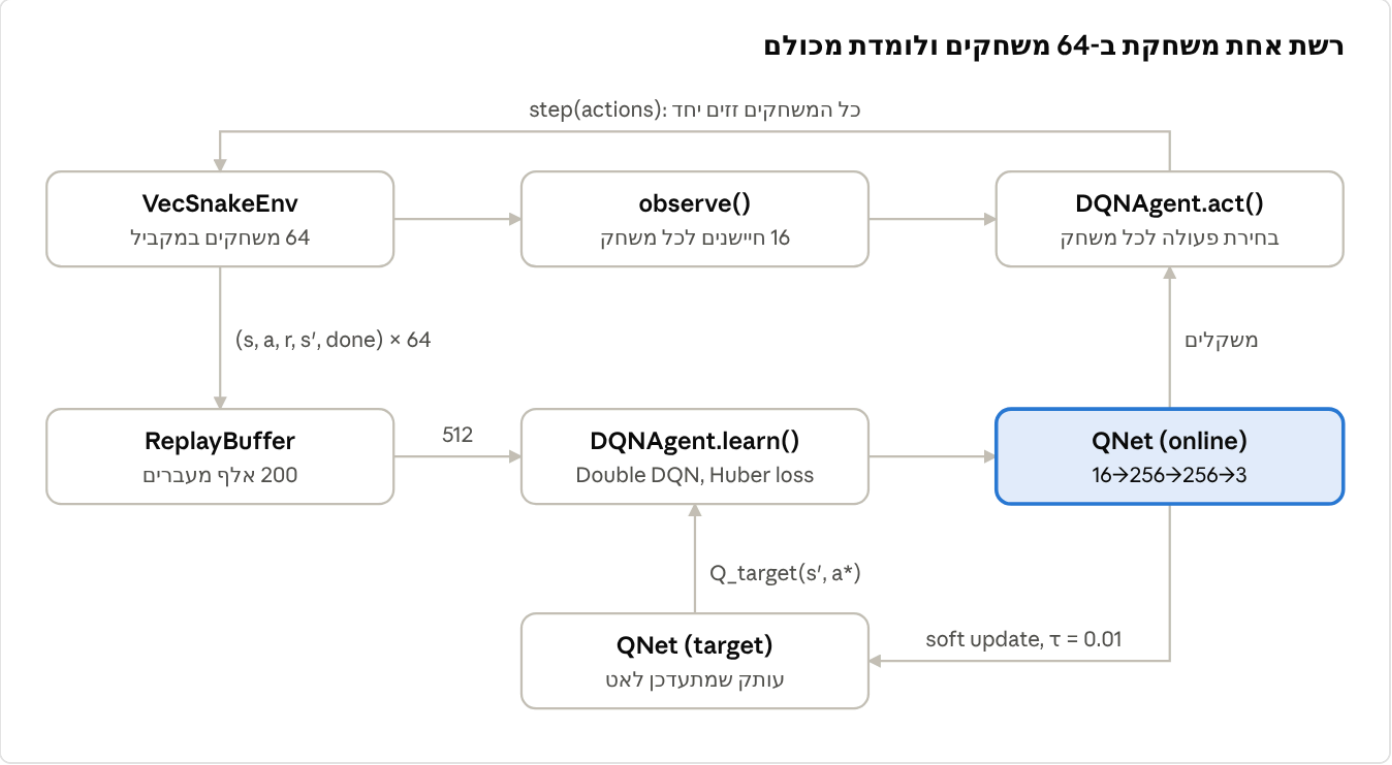
7. **הנחש עיוור לגוף שלו:** החיישנים רואים משבצת אחת קדימה בלבד. כל 500 משחקי ההערכה של המודל המקורי הסתיימו בהתנגשות בגוף, כי הנחש נכנס לכיסים סגורים.

בעיות 1–4 הן בעיות מהירות; בעיות 5–7 פוגעות באיכות הלמידה. המימוש החדש פותר את שתי הקבוצות.

4. ארכיטקטורת המימוש החדש

המימוש החדש מפריד בין שלושה חלקים: הסביבה (חוקי המשחק), הסוכן (הרשת והלמידה), והציור. כך אפשר לאמן בלי מסך כלל, ולצייר רק כשרוצים לצפות.

קובץ	תפקיד
`snake_env.py`	סביבה וקטורית של N משחקים וחישוב החיישנים
`dqn.py`	רשת, זיכרון וסוכן Double DQN
`train.py`	לולאת האימון, הערכה תקופתית ושמירת תוצאות
`evaluate.py`	מדידה של מודל על 500 משחקים, כולל המודל המקורי
`watch.py`	צפייה במודל מאומן במסך מפוצל
`split_view.py`	ציור כמה משחקים בחלון pygame אחד
`compare_runs.py`	גרף השוואה בין ריצות
`model_select.py`	בחירת המודל הכי טוב מתוך `runs/`
`runs/<name>/`	תוצרי כל ריצה: `progress.png`, `eval.csv`, `log.csv`, `last.pth`, `best.pth`



זרימת הנתונים ב-train.py · צעד אחד של לולאת האימון

בכל צעד של הלולאה כל 64 המשחקים זזים יחד, 64 מעברים נכנסים לזיכרון, והרשת מבצעת צעד למידה אחד על 512 מעברים אקראיים. רשת המטרה משמשת רק לחישוב המטרה, ונגררת אחרי הרשת הראשית.

5. הסביבה המקבילית: `snake_env.py`

המחלקה `VecSnakeEnv` מריצה N משחקים במקביל כפעולות על מערכי NumPy, בלי לולאה על המשחקים ובלי ציור. היא מגיעה ל-9,000–5,000 צעדים בשנייה, פי 130–220 מהמקור. החוקים זהים ל-`game.py`: לוח 32×24, נחש באורך 3 פונה ימינה, 10−/10+.

5.1 ייצוג הלוח: "כמה צעדים נשארו"

המערך `body` בגודל (N, 24, 32). הערך בכל משבצת הוא מספר הצעדים שהיא עוד תהיה תפוסה. הראש מקבל את אורך הנחש, והזנב את הערך 1.

- **צעד רגיל:** כל הערכים יורדים ב-1, והזנב מתפנה מעצמו. אין צורך לשמור רשימת משבצות.
- **אכילה:** לא מורידים כלום, והנחש גדל במשבצת אחת.
- **התנגשות בגוף:** בדיקה אחת, `body[y, x] > 0`, במקום חיפוש ברשימה כמו ב-`is_collision` המקורי.

5.2 הפונקציות

פונקציה	מה היא עושה	פרט מימוש חשוב
`__init__(n_envs=64, ...)`	מקצה מערכים לכל המשחקים: לוח, ראש, כיוון, אורך, אוכל, ניקוד, רעב	כל שדה הוא מערך באורך N
`reset(mask)`	מאפס רק את המשחקים שמסומנים ב-mask	משחק שנגמר מתחיל מחדש מיד, ושאר המשחקים ממשיכים
`_place_food(ids)`	אוכל במשבצת פנויה אקראית	רעש אקראי על כל הלוח, 1− על משבצות תפוסות, ו-argmax. בלי רקורסיה כמו במקור
`step(actions)`	מזיז את כל המשחקים צעד אחד	מחזיר תגמולים, סימון סוף וניקוד סופי; מאפס משחקים שנגמרו
`observe()`	מחזיר מטריצה (N, 16) של חיישנים	מפורט בפרק 6
`snapshot(i)`	משבצות הנחש מהראש לזנב, אוכל וניקוד של משחק i	משמש רק לציור; ממיין לפי הערכים ב-`body`

5.3 מה קורה ב-`step`

1. הכיוון החדש מחושב במודולו 4: ישר = 0, ימינה = 1+, שמאלה = 1− (סדר הכיוונים: ימינה, למטה, שמאלה, למעלה).
2. במשחקים שבהם הנחש לא אוכל, כל הלוח יורד ב-1 (הזנב זז). זה קורה **לפני** בדיקת ההתנגשות, כך שמותר להיכנס למשבצת שהזנב עוזב באותו צעד.
3. מוות = קיר, או גוף, או רעב (יותר מ-100×אורך צעדים מאז האוכל האחרון).
4. נחשים חיים מעדכנים ראש ואורך; מי שאכל מקבל אוכל חדש; משחקים שנגמרו מתאפסים.

הסביבה נבדקה מול מימוש רגיל צעד אחרי צעד, על 20,000 צעדים, בלי אף הבדל (לפי ה-README).

הבדל אחד מהמקור: ב-`game.py` המונה `frame_iteration` מתאפס רק בתחילת משחק, ולכן כלל הרעב שם סופר את כל הצעדים במשחק. ב-`snake_env.py` המונה מתאפס בכל אכילה.

6. החיישנים: מה הנחש "רואה"

חמשת החיישנים החדשים הם השינוי שהעלה את הממוצע מ-38 ל-145. עם אותו קוד בדיוק ו-11 חיישנים בלבד, הממוצע נשאר סביב 37.

#	חיישן	ערכים	למה הוא שם
0–2	סכנה ישר / ימינה / שמאלה	0 או 1	מונע מוות מיידי בקיר או בגוף (מקורי)
3–6	כיוון תנועה: שמאלה / ימינה / למעלה / למטה	one-hot	מאפשר לקשר בין פעולה יחסית למיקום האוכל (מקורי)

#	חיישן	ערכים	למה הוא שם
7–10	אוכל משמאל / מימין / למעלה / למטה	0 או 1	מכוון את הנחש לאוכל (מקורי)
11–13	שטח פנוי נגיש אחרי ישר / ימינה / שמאלה	0 עד 1	מזהה מראש כיס שהגוף סוגר (חדש)
14	רעב: צעדים מאז האוכל / מגבלת הרעב	0 עד 1	מאפשר ללמוד שלא להסתובב במעגלים עד מוות מרעב (חדש)
15	אורך הנחש / 768 משבצות	0 עד 1	נחש ארוך צריך לשחק זהיר יותר מנחש קצר (חדש)

6.1 איך מחושב השטח הפנוי (flood fill)

1. בונים מפת משבצות פנויות לכל המשחקים: `body <= 1`. הזנב (ערך 1) נחשב פנוי, כי הוא מתפנה בצעד הבא. הראש מסומן כתפוס.
2. `scipy.ndimage.label` מסמן רכיבי קשירות (שכנים ב-4 כיוונים). הקריאה רצה על מערך תלת-ממדי של כל המשחקים, עם מבנה שכנות שאין בו קשר בין לוחות. כך כל 64 המשחקים מחושבים בקריאה אחת.
3. `np.bincount` סופר את גודל כל רכיב.
4. לכל אחת משלוש המשבצות שליד הראש קוראים את גודל הרכיב שלה, מחלקים במספר המשבצות הפנויות, ושמים 0 אם יש שם סכנה.

דוגמה: אם פנייה ימינה מובילה לכיס של 5 משבצות ופנייה שמאלה לשטח של 600, חיישן 12 קרוב ל-0 וחיישן 13 קרוב ל-1. הרשת לומדת מהר שכיס קטן מוביל למוות.

במקור החיישנים היו בוליאניים בלבד. כאן כל החיישנים מנורמלים לטווח 0–1, כך שאף חיישן לא שולט ברשת בגלל סדר גודל. הדגל `--features 11` ב-`train.py` משתמש רק ב-11 הראשונים, לניסוי ההשוואה.

7. הסוכן: `dqn.py`

`dqn.py` מממש Double DQN עם רשת מטרה וזיכרון NumPy, ומחשב את כל ה-batch בפעולה אחת, בלי לולאת Python.

7.1 המחלקות והפונקציות

מחלקה / פונקציה	מה היא עושה
`QNet`	רשת של 16 ← 256 ← 256 ← 3 עם ReLU. הפלט: ערך Q לכל פעולה
`ReplayBuffer.__init__`	מקצה מראש חמישה מערכים של 200 אלף שורות: s, a, r, s′, done
`ReplayBuffer.add_batch`	מוסיף 64 מעברים בבת אחת, כמערך מעגלי (הישן ביותר נדרס)
`ReplayBuffer.sample`	בוחר 512 אינדקסים אקראיים ומחזיר טנזורים
`DQNAgent.__init__`	יוצר רשת ראשית ורשת מטרה זהות, Adam (lr = 0.001), וזיכרון

מחלקה / פונקציה	מה היא עושה
`DQNAgent.act(obs, ε)`	ε-greedy לכל המשחקים יחד: מעבר אחד ברשת והגרלת מסכת חקירה
`DQNAgent.learn()`	צעד למידה אחד (פירוט למטה)
`DQNAgent.save`	שומר משקלים, היפר-פרמטרים ומטא-דאטה (צעדים, שיא, ציון הערכה)
`DQNAgent.load_policy`	טוען קובץ ובונה ממנו רשת בגודל הנכון (11 או 16 כניסות)

7.2 צעד למידה ב- `learn()`

1. דוגמים 512 מעברים מהזיכרון.
2. מחשבים Q(s, a) של הפעולה שבוצעה, בעזרת `gather`.
3. הרשת הראשית בוחרת $a^* = \text{argmax}\, Q(s')$, ורשת המטרה מעריכה אותה (Double DQN).
4. מטרה: $r + 0.95 \cdot (1 - \text{done}) \cdot Q_{\text{target}}(s', a^*)$.
5. **Huber loss** (`smooth_l1_loss`): כמו MSE לשגיאות קטנות, ולינארי לשגיאות גדולות. כך תגמול של 10− לא יוצר גרדיאנט עצום.
6. **Gradient clipping** לנורמה 10, נגד קפיצות.
7. עדכון רך של רשת המטרה, $\tau = 0.01$.

7.3 השוואה למקור

נושא	מקור (`model.py` , `agent.py`)	חדש (`dqn.py`)
רשת	3→256→11	3→256→256→16
מטרה	אותה רשת, max רגיל	רשת מטרה, Double DQN
פונקציית הפסד	MSE	Huber + clipping
חישוב ה-batch	לולאת Python על 1,000 דוגמאות	פעולה וקטורית אחת על 512
זיכרון	`deque` של 100 ,tuples אלף	מערכי NumPy, 200 אלף
תדירות למידה	צעד קצר בכל מהלך + צעד ארוך בסוף משחק	צעד אחד על batch בכל מהלך של 64 משחקים
ε	80 – מספר משחקים; אפס אחרי 80 משחקים	יורד לינארית מ-1.0 ל-0.001 לאורך 25% מהצעדים

ε מסתיים ב-0.001 ולא ב-0.01: נחש של 100 משבצות מבצע אלפי מהלכים במשחק, וב-0.01 כמעט בטוח שמהלך אקראי יהרוג אותו.

8. לולאת האימון: `train.py`

`train.py` מאמן מיליון צעדים כברירת מחדל, ב-15,625 איטרציות של 64 משחקים, ובוחר את המודל הטוב לפי הערכה בלי אקראיות.

8.1 הפונקציות

פונקציה	מה היא עושה
`parse()`	קוראת את פרמטרי שורת הפקודה (טבלה בהמשך)
`greedy_eval(agent, nf, n_games)`	מריצה 50 משחקים במקביל בלי חקירה, עם seed קבוע (999), ומחזירה את ממוצע המשחק הראשון של כל סביבה
`save_plot(games_log, path)`	שומרת `progress.png`: ניקוד לכל משחק וממוצע נע של 200 משחקים, לפי מספר משחקים ולפי זמן
`main()`	לולאת האימון

8.2 מה קורה בכל איטרציה של `main()`

1. מחשבים ε לפי ההתקדמות.
2. `agent.act` בוחר 64 פעולות.
3. `env.step` מזיז את כל המשחקים, ו-`env.observe` מחזיר את המצבים החדשים.
4. 64 המעברים נכנסים לזיכרון.
5. אחרי 5,000 מעברים ראשונים (warmup), צעד למידה אחד.
6. משחקים שנגמרו נרשמים ב-`log.csv`.
7. כל 100 אלף צעדים: `greedy_eval`, שורה ב-`eval.csv`, ואם זה שיא חדש, שמירת `best.pth`.
8. עם `--view`, ציור של המסך המפוצל כל 4 איטרציות. האימון לא מחכה למסך.
9. כל 10 שניות, שורת סטטוס במסוף: צעדים בשנייה, ממוצע, שיא, ε.

בסוף האימון רצה הערכה אחרונה, ונשמרים `last.pth` ו-`progress.png`.

8.3 פרמטרים

פרמטר	ברירת מחדל	משמעות
`--envs`	64	מספר משחקים במקביל
`--steps`	1,000,000	סך כל הצעדים, בכל המשחקים יחד
`--eps-end` / `--eps-start`	0.001 / 1.0	טווח החקירה

פרמטר	ברירת מחדל	משמעות
`--eps-frac`	0.25	חלק האימון שבו ε יורד
`--lr`	0.001	קצב למידה של Adam
`--gamma`	0.95	מקדם הנחה
`--batch`	512	גודל מנה למידה
`--warmup`	5,000	מעברים לפני תחילת הלמידה
`--eval-games` / `--eval-every`	50 / 100,000	תדירות וגודל ההערכה
`--features`	16	11 = החיישנים המקוריים בלבד
`--show` / `--view`	כבוי / 9	מסך מפוצל בזמן אימון
`--name`	חותמת זמן	שם התיקייה ב- `runs/`

8.4 למה בוחרים מודל לפי הערכה בלי אקראיות

ממוצע האימון כולל מהלכים אקראיים, ולכן הוא נמוך בהרבה מהיכולת האמיתית: כ-50 באימון, לעומת 105–145 בהערכה. הוא גם מערבב משחקים ששוחקו על ידי גרסאות שונות של הרשת. הערכה עם seed קבוע מודדת רשת אחת בתנאים שווים.

הערה: ה-docstring בראש `train.py` לא מעודכן. הוא מזכיר 1.5 מיליון צעדים ובחירה לפי ממוצע 200 משחקים, אבל הקוד עצמו עובד כפי שמתואר כאן.

9. למה 64 משחקים במקביל, ואיך זה תורם

זה **סוכן אחד שמשחק ב-64 עותקים של הסביבה**, לא 64 סוכנים נפרדים. יש רשת אחת, זיכרון אחד ואופטימייזר אחד. כל המשחקים משתמשים באותם משקלים בדיוק, וכל מה שקורה בכל משחק מלמד את אותה רשת. בספרות קוראים לזה **vectorized environments**.

9.1 מה זה תורם

1. **מהירות.** במקום משחק אחד עם ציור בכל צעד (39 צעדים בשנייה), פקודת NumPy אחת מזיזה 64 נחשים. הרשת מופעלת פעם אחת על מטריצה של 64 מצבים, ולא 64 פעמים. העלות של צעד אחד מתחלקת על 64 מעברים.

2. **יותר נתונים לכל צעד למידה.** כל צעד למידה מגיע אחרי 64 מעברים חדשים. הזיכרון מתמלא מהר, והרשת לא מתאמנת שוב ושוב על אותן דוגמאות ישנות.

3. **גיוון בנתונים.** בכל רגע 64 המשחקים נמצאים בשלבים שונים: נחש קצר בהתחלה, נחש ארוך לקראת הסוף, נחש רעב. המעברים בזיכרון פחות מתואמים זה לזה, וזה בדיוק מה ש-Experience Replay צריך כדי שהאימון יהיה יציב.

4. **הפרדה מהציור.** המסך מצייר רק כמה משחקים ורק כל כמה צעדים, ולכן אפשר לצפות באימון בלי להאט אותו.

9.2 השפעה במספרים

	מקור	חדש, 64 משחקים
צעדי משחק בשנייה	כ-39	5,400–3,200; כ-9,000–5,000 על הסביבה לבדה (README) כולל למידה והערכות (`log.csv`)
זמן עד הרמה של המודל המקורי, 11 חיישנים	כשעה וחצי	כ-30 שניות
מיליון צעדים	כ-7 שעות (חישוב לפי 39 בשנייה)	3 עד 5 דקות

9.3 למה לא כמה סוכנים נפרדים

כמה סוכנים עם רשתות נפרדות היו מחלקים ביניהם את הנתונים, וכל אחד היה לומד לאט יותר. ב-Snake אין אינטראקציה בין נחשים, ולכן אין סיבה למידת רב-סוכנים. המטרה היא רשת אחת שמשחקת טוב, ו-64 המשחקים הם פשוט דרך מהירה לאסוף לה ניסיון. שיטות כמו A3C מריצות כמה "עובדים" מקביליים מאותה סיבה, וגם שם כולם מעדכנים רשת משותפת אחת.

מספר המשחקים קביע ב- `--envs` . יותר משחקים נותנים יותר צעדים בשנייה, אבל פחות צעדי למידה לכל מעבר. לכן יש נקודת איזון, ו-64 הוא ערך ברירת המחדל שנבחר.

10. כלי הערכה וצפייה

חמישה קבצי עזר מודדים, משווים ומציגים מודלים מאומנים. כולם משתמשים באותה סביבה, כך שהמדידות הן בתנאים זהים.

קובץ	פונקציות	מה הוא עושה
`evaluate.py`	`run_games(policy, n_games)` , `main()`	500 משחקים בלי אקראיות (100 במקביל, seed 123). מדפיס ממוצע, חציון, מקסימום, מינימום וסיבת מוות: קיר, גוף או רעב. עם `--legacy` הוא טוען את המודל המקורי ומזין לו את 11 החיישנים הראשונים, שהם באותו סדר ובאותה משמעות
`watch.py`	`main()`	מציג מודל מאומן משחק בכמה משחקים במקביל (ברירת מחדל 9), עם ממוצע ושיא בשורת הכותרת. `--fps` קובע את המהירות
`split_view.py`	`SplitView.__init__` , `handle_events` , `draw` , `close`	מחלק חלון pygame אחד לרשת של לוחות ומצייר כל משחק בעזרת `env.snapshot` . `handle_events` מחזיר False כשסוגרים את החלון

קובץ	פונקציות	מה הוא עושה
`compare_runs.py`	`main()`	קורא את `eval.csv` של כמה ריצות ומצייר את עקומות ההערכה לפי צעדים ולפי זמן, עם קו ייחוס של המודל המקורי (33.5). שומר `runs/comparison.png`
`model_select.py`	`best_model()`	עובר על כל `runs/*/best.pth` ומחזיר את זה עם ציון ההערכה הגבוה ביותר

סיבת המוות מחושבת לפני הצעד: אם המשבצת הבאה מחוץ ללוח זה קיר; אם חיישן הרעב במקסימום זה רעב; אחרת זו התנגשות בגוף.

11. ניסויים ותוצאות

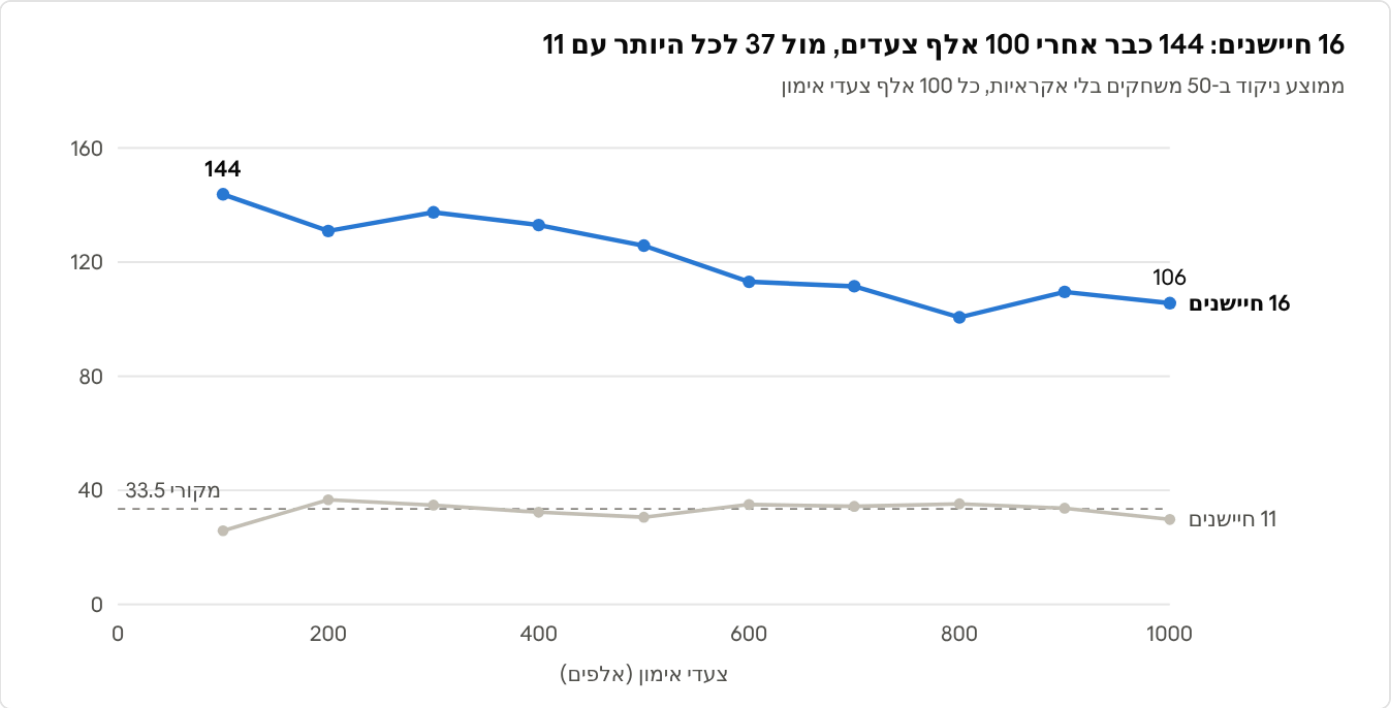
המודל עם 16 חיישנים מנצח בפער גדול: ממוצע של 145.4 ושיא של 203 ב-500 משחקי הערכה. הסיבה היא החיישנים, לא המהירות ולא אורך האימון.

11.1 הערכה סופית של המודלים

500 משחקים בלי אקראיות עם `evaluate.py`, באותם תנאים לכל המודלים (הנתונים מה-README):

מודל	זמן אימון	ממוצע	שיא
מקורי, 11 חיישנים (`model/model.pth`)	כשעה וחצי	33.5	84
קוד חדש, 11 חיישנים (`runs/feat11/best.pth`)	30 שניות	37.8	77
קוד חדש, 16 חיישנים (`runs/feat16/best.pth`)	**20 שניות**	**145.4**	**203**

11.2 הערכה לאורך האימון



מיליון צעדי אימון לכל ריצה · runs/feat16/eval.csv, runs/feat11/eval.csv

שתי הריצות זהות בכל הפרמטרים חוץ ממספר החיישנים. עם 11 חיישנים הסוכן תקוע סביב המודל המקורי לאורך כל האימון. עם 16 הוא מגיע לשיא בהערכה הראשונה, ומשם יורד ל-106.

11.3 נתוני הריצות ב-`runs/`

ריצה	חיישנים	צעדים	זמן	משחקים	ממוצע באימון, 200 אחרונים	שיא באימון
`feat16`	16	1,000,000	5.1 דקות	2,308	93.3	175
`feat11`	11	1,000,000	3.1 דקות	3,327	31.1	70
`main`	16	4,000,000	24.6 דקות	11,519	47.0	134

הריצה `main` נעשתה בגרסה מוקדמת של הקוד: ε סופי של 0.01, בלי קובץ `eval.csv`, ו-`best.pth` נבחר לפי ממוצע האימון. אימון ארוך פי 4 לא הביא בה לממוצע אימון גבוה יותר מזה של feat16.

12. ממצאים ודיון

1. **הייצוג חשוב יותר מהמהירות.** אותו קוד בדיוק, עם 11 חיישנים במקום 16, נתקע סביב 37 גם אחרי מיליון צעדים. נחש שרואה משבצת אחת קדימה לא יכול לזהות מלכודת, ושום כמות של אימון לא מפצה על מידע שחסר במצב.

2. **המהירות קיצרה את הניסויים.** רמה של המודל המקורי מושגת ב-30 שניות במקום שעה וחצי. זה מה שאפשר להריץ השוואות כמו 11 מול 16 חיישנים בכמה דקות.

3. **אימון ארוך יותר פגע.** עם 16 חיישנים הציון ירד מ-144 ל-106 במהלך מיליון צעדים. ההסבר הסביר הוא catastrophic forgetting: כשהזיכרון מתמלא במצבים של נחש ארוך, הרשת מאבדת חלק ממה שעבד קודם. לכן בחירת `best.pth` לפי הערכה היא חיונית, ולא רק שמירת המודל האחרון.

4. **ממוצע האימון מטעה.** בריצת feat16 ממוצע 200 המשחקים האחרונים באימון הוא 93, לעומת 145 בהערכה. הפער נובע ממהלכים אקראיים ומשינוי הרשת לאורך האימון.

5. **הסוכן הטוב נמצא מוקדם מאוד.** `best.pth` של feat16 נשמר אחרי 100 אלף צעדים בלבד, 793 משחקים ו-20 שניות. עם חיישנים טובים, הבעיה קלה ללמידה.

13. מגבלות וכיווני המשך

המגבלה העיקרית היא נחש ארוך: לפי ה-README, 94% ממשחקי ההערכה עדיין נגמרים בהתנגשות בגוף, כשהנחש באורך 100 ויותר על לוח של 768 משבצות.

מגבלות נוספות:

- החיישנים נבחרו ידנית. הרשת לא רואה את הלוח עצמו.
- כל ריצה נעשתה עם seed אחד, ולכן אין הערכה של השונות בין ריצות.
- ההערכה במהלך האימון מבוססת על 50 משחקים בלבד, ולכן רועשת.

כיווני המשך:

- **רשת קונבולוציה (CNN)** שמקבלת את הלוח כולו, במקום חיישנים ידניים.
- **Prioritized Experience Replay** ו-**n-step returns**, ללמידה מהירה יותר ממעברים חשובים.
- **קצב למידה יורד או עצירה מוקדמת** לפי הערכה, נגד הירידה באימון ארוך.
- **חיישן "הזנב נגיש"**: נחש שיכול לעקוב אחרי הזנב שלו לא נתקע.
- כמה seeds לכל ניסוי, עם ממוצע וסטיית תקן.

14. הוראות הרצה

נדרשים Python 3.11 והספריות numpy, scipy, torch, pygame ו-matplotlib.

```
pip install -r requirements.txt

python train.py                          # אימון: 64 משחקים במקביל, בלי גרפיקה
python train.py --view                   # אימון עם מסך מפוצל של 9 משחקים
python train.py --features 11 --name feat11   # ניסוי ההשוואה

python watch.py                          # צפייה במודל הכי טוב
python watch.py --show 16 --fps 40

python evaluate.py                       # 500 משחקים: ממוצע, חציון, שיא, סיבת מוות
python evaluate.py --legacy model/model.pth   # המודל המקורי
python compare_runs.py feat16 feat11   # גרף השוואה

python agent.py                          # המימוש המקורי
```

כל אימון נשמר ב-`runs/<name>/`. הסקריפטים `watch.py` ו-`evaluate.py` בוחרים לבד את המודל עם ציון ההערכה הגבוה ביותר, אלא אם מעבירים `--model`.